VoxelTracer — In-Depth Code Explanation & Walkthrough

This document provides a comprehensive code-level explanation of every file in the VoxelTracer system. It is structured for deep understanding of core concepts, algorithms, GPU/CPU utilisation, and design patterns.

Table of Contents

1. System Overview & Data Flow
2. File: VoxelTracerSystem.cs
3. File: VoxelTracerCore.compute
4. File: VoxelTracerRayMarch.compute
5. File: VoxelTracerCamera.cs
6. File: VoxelComposite.shader
7. File: VoxelTracerComposite.shader
8. File: VoxelDynamic.cs
9. File: VoxelNormalGizmos.cs
10. File: VoxelSliceViewer.cs
11. File: RandomRotator.cs
12. Cross-Cutting Concepts
13. References & Plagiarism Analysis

---

1. System Overview & Data Flow

The VoxelTracer is a real-time GPU voxelisation and rendering system for Unity's built-in render pipeline. It converts arbitrary scene geometry (meshes, skinned meshes, terrains) into a 3D voxel grid every frame on the GPU, then visualises that grid using a full-screen DDA ray march composited over the camera's normal scene output.

High-Level Pipeline

```
┌──────────────────────────────────────────────────────────────────────┐
│                        CPU SIDE (C#)                                 │
│                                                                      │
│  VoxelTracerSystem                                                   │
│  ├─ BuildTriangleLists()         — extract triangles from scene      │
│  ├─ ComputeBoundsFromBothLists() — compute world AABB                │
│  ├─ ComputeGridSize()            — compute grid dimensions           │
│  ├─ AllocateResources()          — create GPU buffers & textures     │
│  ├─ UploadStaticTriangles()      — CPU → GPU once                    │
│  ├─ RebuildStatic()              — voxelise static geometry once     │
│  └─ VoxelizeFrame()              — per-frame GPU dispatch chain      │
│        │                                                             │
│        ▼                                                             │
│  ┌──────────────────────────────────────────────────────────────┐    │
```

```
│  │             GPU SIDE (Compute Shaders)                    │     │
│  │                                                           │     │
│  │  VoxelTracerCore.compute                                  │     │
│  │  ├─ CopyStaticToWorking  (linear copy)                    │     │
│  │  ├─ ClearFlood           (clear flood buffer)             │     │
│  │  ├─ Surface              (SAT triangle-AABB test)         │     │
│  │  ├─ SweepFill × 3 axes  (bidirectional flood fill)      │     │
│  │  ├─ BuildTexture         (buffer → RWTexture3D<float>)    │     │
│  │  ├─ BlurFill             (3×3×3 box blur)                 │     │
│  │  └─ ComputeNormals       (gradient from blurred fill)     │     │
│  │                                                           │     │
│  │  Outputs: _FillTex (3D), _NormalsTex (3D)                 │     │
│  └──────────────────────────┬──────────────────────────────┘     │
│                             │                                      │
│  VoxelTracerCamera                                                 │
│  ├─ Dispatch RayMarch (per screen pixel)                           │
│  └─ Composite over scene (Blit with shader)                        │
│        │                                                           │
│        ▼                                                           │
│  ┌──────────────────────────────────────────┐                    │
│  │  VoxelTracerRayMarch.compute              │                    │
│  │  └─ RayMarch kernel (DDA traversal)       │                    │
│  │       Output: _ColorOut (2D RGBA float)   │                    │
│  └──────────────────────────────────────────┘                    │
│        │                                                           │
│        ▼                                                           │
│  ┌──────────────────────────────────────────┐                    │
│  │  VoxelComposite.shader                    │                    │
│  │  └─ alpha-over blend scene + voxels       │                    │
│  └──────────────────────────────────────────┘                    │
└────────────────────────────────────────────────────────────────────┘
```

Key Architectural Decisions

1. **Dual-buffer static/dynamic split** — Static geometry is SAT-voxelised once and cached in `_staticVoxelBuf`. Per-frame, this cache is copied into the working buffer, then only dynamic triangles are SAT-tested on top.
2. **Structured buffers for voxelisation, textures for rendering** — Voxelisation uses `RWStructuredBuffer<uint>` (atomic-safe), then the result is copied into `RWTexture3D<float>` for the ray march's `Texture3D.Load()` lookups.
3. **DDA face normals for rendering** — Instead of reading from a normal texture, the ray march derives normals from which voxel face the ray entered. This gives clean axis-aligned normals (6 directions) with zero memory overhead.
4. **Blurred gradient normals for debug visualisation** — A separate blur→gradient pipeline in VoxelTracerCore computes smooth normals into `_NormalsTex` for the VoxelNormalGizmos debug tool.

---

2. File: VoxelTracerSystem.cs

General Purpose

This is the **main controller MonoBehaviour** — the orchestrator of the entire voxelisation pipeline. It handles:

- CPU-side triangle extraction from scene geometry (meshes, skinned meshes, terrains)
- Bounds computation and grid sizing
- GPU resource allocation (buffers, textures)
- Dispatching all compute shader kernels in the correct order
- The static/dynamic caching architecture

Function-by-Function Walkthrough

Inspector Fields

```
public ComputeShader coreCS;
```

A Unity `ComputeShader` asset reference. The inspector slot expects `VoxelTracerCore.compute` which contains all 8 GPU kernels for voxelisation. Unity's `ComputeShader` is a C# wrapper over a DirectX/Vulkan/Metal compute pipeline compiled from HLSL `.compute` files.

```
public BoundsMode boundsMode = BoundsMode.AutoFitScene;
```

Enum controlling whether the voxel grid's world-space AABB is manually specified or auto-computed from all scene geometry. `AutoFitScene` iterates every triangle vertex to find the tightest enclosing box, then adds `autoFitPadding` world units on each side.

```
[Min(0.01f)] public float voxelSize = 0.25f;
```

World-space edge length of one cubic voxel. At 0.25, a 20-unit scene produces an 80×80×80 grid (512K voxels). This is the primary quality/performance knob.

```
[Range(1, 4)] public int fillSweepRounds = 1;
```

How many full 6-direction sweep-fill passes to run. One pass handles most convex and simple concave geometry. Highly concave geometry (e.g., a room with a narrow doorway) may need 2+ to fully flood the exterior.

```
[Range(32, 512)] public int maxVoxelsPerAxis = 256;
[Min(1)] public float maxVoxelCountMillions = 32f;
```

Safety caps preventing runaway GPU allocations. If the computed grid dimensions exceed these, the grid is uniformly scaled down using a cube-root formula.

Struct: `Tri`

```
[StructLayout(LayoutKind.Sequential)]
struct Tri
{
    public Vector3 a, b, c, n;
}
```

Triangle data transferred to the GPU. `StructLayout(LayoutKind.Sequential)` guarantees the C# struct's memory layout matches the HLSL struct `{ float3 a, b, c, normal; }` — 4 × `float3` = 48 bytes. Without this attribute, the CLR could reorder fields or add padding, causing data corruption when read on the GPU.

Why `LayoutKind.Sequential` matters: `ComputeBuffer.SetData()` does a raw `memcpy` from the C# array to GPU memory. The GPU side expects fields at exact byte offsets: `a` at 0, `b` at 12, `c` at 24, `n` at 36. If C# rearranged any field, the GPU would interpret garbage values.

`OnEnable()` / `OnDisable()` / `LateUpdate()`

```
void OnEnable()
{
    if (coreCS == null) return;
    CacheKernels();
    RebuildStatic();
    VoxelizeFrame();
}
```

`OnEnable()` — Called by Unity when the component is enabled (including initial activation). It caches kernel indices (string lookups are expensive so they're done once), then performs the one-time static voxelisation and the first frame's full pipeline.

`OnDisable()` — Releases all GPU resources. This is critical for preventing GPU memory leaks — Unity does not garbage-collect `ComputeBuffer` or `RenderTexture` objects automatically.

`LateUpdate()` — Called after all `Update()` methods. Using `LateUpdate` ensures that dynamic objects have already moved/animated this frame before triangles are extracted.

```
void LateUpdate()
{
    if (coreCS == null) return;
    if (_staticDirty)
        RebuildStatic();
    VoxelizeFrame();
}
```

The `_staticDirty` flag allows external code (or the context menu) to request a full static re-bake. Normally, only `VoxelizeFrame()` runs each frame.

`RebuildStatic()`

```
void RebuildStatic()
{
    if (!_kernelsCached) CacheKernels();
    _staticDirty = false;

    BuildTriangleLists();  // populates _staticTriList and _dynamicTriList

    int totalTris = _staticTriList.Count + _dynamicTriList.Count;
    if (totalTris == 0) { _nx = _ny = _nz = 0; return; }

    ComputeBoundsFromBothLists(out Vector3 mn, out Vector3 mx);
    _activeMin = mn; _activeMax = mx;

    ComputeGridSize(mn, mx, out int gx, out int gy, out int gz);
    _nx = gx; _ny = gy; _nz = gz;
    _totalVoxels = gx * gy * gz;

    AllocateResources(gx, gy, gz);
    UploadStaticTriangles();
    // ... dispatch SAT on static triangles ...
    CopyWorkingToStaticCache();
}
```

This method:

1. Calls `BuildTriangleLists()` which scans ALL scene objects (static + dynamic) — both lists are needed to compute bounds that encompass everything.
2. Computes the world-space AABB from both lists. Even dynamic objects contribute to the grid bounds so they're always inside the volume.
3. Computes grid dimensions with safety capping.
4. Allocates all GPU resources (destroys previous ones if they exist).
5. Uploads static triangles to `_staticTriBuffer`, dispatches `Clear` + `Surface` kernels, then snapshots the resulting `_voxelBuffer` into `_staticVoxelBuf`.

`CopyWorkingToStaticCache()` — The Static Bake Snapshot

```
void CopyWorkingToStaticCache()
{
    var data = new uint[_totalVoxels];
    _voxelBuffer.GetData(data);
    _staticVoxelBuf.SetData(data);
}
```

This performs a GPU → CPU → GPU round-trip (`GetData` reads from `_voxelBuffer` into a C# array, `SetData` writes it into `_staticVoxelBuf`). This is intentionally acceptable because it only runs once (or on explicit `MarkStaticDirty()`). A more optimised approach would use `ComputeBuffer.CopyCount` or a copy kernel, but the one-time cost is negligible.

`VoxelizeFrame()` — The Per-Frame Fast Path

This is the **hot path** that runs every frame. It executes 7 GPU dispatches:

```
void VoxelizeFrame()
{
    // 1) Copy cached static voxels → working buffer
    coreCS.SetBuffer(KCopyStaticToWorking, "_VoxelBuffer", _voxelBuffer);
    coreCS.SetBuffer(KCopyStaticToWorking, "_StaticVoxelBuffer", _staticVoxelBuf);
    DispatchLinear(KCopyStaticToWorking, _totalVoxels);

    // 2) Clear flood buffer
    coreCS.SetBuffer(KClearFlood, "_FloodBuffer", _floodBuffer);
    DispatchLinear(KClearFlood, _totalVoxels);

    // 3) Surface SAT — dynamic triangles only
    if (_dynamicTriCount > 0)
    {
        coreCS.SetBuffer(KSurface, "_Tris", _dynamicTriBuffer);
        DispatchLinear(KSurface, _dynamicTriCount);
    }

    // 4) Sweep flood fill (3 axes × N rounds)
    if (fillVolume)
    {
        for (int round = 0; round < fillSweepRounds; round++)
        {
            DispatchSweep(0, gy, gz);  // sweep X, dispatch over Y×Z plane
            DispatchSweep(1, gx, gz);  // sweep Y, dispatch over X×Z plane
            DispatchSweep(2, gx, gy);  // sweep Z, dispatch over X×Y plane
        }
    }

    // 5) Build fill texture
    Dispatch3D(KBuildTexture, gx, gy, gz);

    // 6) Blur fill texture
    Dispatch3D(KBlurFill, gx, gy, gz);

    // 7) Compute normals from blurred fill
    Dispatch3D(KComputeNormals, gx, gy, gz);
}
```

Key insight: Step 1 restores the static voxels in O(N) where N = total voxels, avoiding the expensive SAT re-test for all static triangles. Step 3 only runs SAT for the (usually much smaller) set of dynamic triangles. The

union happens implicitly — `InterlockedOr` in the Surface kernel writes `1` bits on top of the static cache's `1` bits.

Triangle Extraction: `BuildTriangleLists()` and `BuildDynamicTriangleList()`

`BuildTriangleLists()` runs during static rebuild and populates BOTH `_staticTriList` and `_dynamicTriList`. It scans all scene objects:

```
var filters = FindObjectsByType<MeshFilter>(FindObjectsSortMode.None);
```

Unity API: `FindObjectsByType<T>(FindObjectsSortMode)` — This is the modern (Unity 2022.2+) replacement for the deprecated `FindObjectsOfType<T>()`. The `FindObjectsSortMode.None` parameter means results are returned in no particular order, which is faster than `InstanceID` sorting. It returns all active loaded objects of type `T`.

Classification logic:

- `MeshFilter` with `VoxelDynamic` component → dynamic list
- `MeshFilter` without `VoxelDynamic` → static list
- `SkinnedMeshRenderer` → always dynamic (animation changes every frame)
- `Terrain` → always static

`BuildDynamicTriangleList()` is the **lightweight per-frame** version. It only queries `VoxelDynamic` objects and skinned meshes — avoiding the full scene scan.

`AppendMesh()`

```
void AppendMesh(Mesh mesh, Matrix4x4 l2w, List<Tri> target)
{
    var verts = mesh.vertices;     // returns a COPY of the vertex array
    var tris = mesh.triangles;     // returns a COPY of the index array
    // ...
    Vector3 a = l2w.MultiplyPoint3x4(verts[i0]);
```

Unity API: `mesh.vertices` — Returns a *new* `Vector3[]` array every call (it's a property that copies the native buffer). This is why the result is stored locally — calling it in a loop would be $O(n^2)$.

Unity API: `Matrix4x4.MultiplyPoint3x4()` — Transforms a point by a 4x4 matrix, treating the last row as `(0,0,0,1)`. This is faster than `MultiplyPoint()` (which handles perspective projections) and is correct for affine transforms (translation, rotation, scale).

```
Vector3 n = Vector3.Cross(b - a, c - a);
float len2 = n.sqrMagnitude;
if (len2 < 1e-20f) continue;   // skip degenerate triangles
n /= Mathf.Sqrt(len2);
```

Degenerate triangles (zero area) are discarded. The threshold `1e-20` is on the *squared* magnitude, which corresponds to triangles with edge products smaller than ~1e-10 units. This prevents division-by-zero and avoids wasting GPU cycles on invisible geometry.

`AppendTerrain()`

```
void AppendTerrain(Terrain terrain, int step, List<Tri> target)
{
    float[,] heights = td.GetHeights(0, 0, hRes, hRes);
```

Unity API: `TerrainData.GetHeights(x, y, width, height)` — Returns a 2D float array where values are normalised [0,1] relative to the terrain's height setting (`TerrainData.size.y`). The indexing is `[z, x]` (row-major, Z-first), which is why the code uses `heights[z0, x0]`.

Unity API: `Terrain.terrainData.heightmapResolution` — The number of height samples along each edge. A 513 heightmap has 512×512 quads. The `step` parameter sub-samples this to reduce triangle count.

The terrain triangulation creates both a **top surface** (from the heightmap) and a **bottom surface** (flat plane at `tPos.y` with reversed winding). The bottom surface is essential for the sweep-fill algorithm — without it, the terrain would be an open surface and the flood fill would mark the underground as "outside", preventing volume fill below the terrain.

`HToW()` — Height-to-World Conversion

```
static Vector3 HToW(int x, int z, float h, int hRes, Vector3 tPos, Vector3 tSize)
{
    float fx = (float)x / (hRes - 1);
    float fz = (float)z / (hRes - 1);
    return new Vector3(tPos.x + fx * tSize.x,
                       tPos.y + h * tSize.y,
                       tPos.z + fz * tSize.z);
}
```

Converts heightmap grid coordinates to world position. `hRes - 1` is used because `hRes` samples span `hRes - 1` intervals across `tSize.x` meters.

Bounds Computation

```
void ComputeBoundsFromBothLists(out Vector3 mn, out Vector3 mx)
{
    mn = new Vector3(float.MaxValue, float.MaxValue, float.MaxValue);
    mx = new Vector3(float.MinValue, float.MinValue, float.MinValue);
    ExpandBounds(_staticTriList, ref mn, ref mx);
    ExpandBounds(_dynamicTriList, ref mn, ref mx);
    Vector3 pad = Vector3.one * autoFitPadding;
```

```
    mn -= pad; mx += pad;
}
```

Iterates every triangle vertex to find the axis-aligned bounding box. Both lists are included so the grid always encompasses dynamic objects (preventing them from falling outside the volume).

The padding prevents surface voxels from touching the grid boundary, which would cause flood-fill artifacts (the boundary is where "outside" propagation starts — surface voxels at the boundary would block the flood incorrectly).

Grid Sizing with Safety Capping

```
void ComputeGridSize(Vector3 mn, Vector3 mx, out int gx, out int gy, out int gz)
{
    Vector3 size = mx - mn;
    gx = Mathf.Max(1, Mathf.CeilToInt(size.x / voxelSize));
    // ... clamp to maxVoxelsPerAxis ...

    long total = (long)gx * gy * gz;
    long budget = (long)(maxVoxelCountMillions * 1_000_000);
    if (total > budget)
    {
        float scale = Mathf.Pow((float)budget / total, 1f / 3f);
        gx = Mathf.Max(1, Mathf.FloorToInt(gx * scale));
        // ...
    }
}
```

The cube-root scaling formula `Pow(budget/total, 1/3)` uniformly reduces all three dimensions to stay within the voxel budget while maintaining the aspect ratio. This prevents situations where one axis is disproportionately reduced.

GPU Resource Allocation: `AllocateResources()`

```
_voxelBuffer = new ComputeBuffer(totalVoxels, sizeof(uint));
_floodBuffer = new ComputeBuffer(totalVoxels, sizeof(uint));
_staticVoxelBuf = new ComputeBuffer(totalVoxels, sizeof(uint));
```

Unity API: `ComputeBuffer(count, stride)` — Allocates GPU memory for a structured buffer. `sizeof(uint)` = 4 bytes per element. For a 128^3 grid (2M voxels), each buffer is ~8 MB.

```
_fillTex = new RenderTexture(gx, gy, 0, RenderTextureFormat.RFloat)
{
    dimension = UnityEngine.Rendering.TextureDimension.Tex3D,
    volumeDepth = gz,
    enableRandomWrite = true,
```

```
    useMipMap = false,
    autoGenerateMips = false,
    wrapMode = TextureWrapMode.Clamp,
    filterMode = FilterMode.Point
};
_fillTex.Create();
```

Unity API: `RenderTexture` **with** `TextureDimension.Tex3D` — Creates a 3D texture. Key properties:

- `volumeDepth = gz` sets the Z dimension (the constructor's `gx` and `gy` set X and Y, but `depth: 0` in the constructor refers to the *depth buffer bit count*, NOT the 3D Z size — a common Unity confusion).
- `enableRandomWrite = true` makes it bindable as `RWTexture3D` in compute shaders. Without this, it can only be read.
- `RenderTextureFormat.RFloat` = single-channel 32-bit float (4 bytes/voxel).
- `FilterMode.Point` = nearest-neighbour sampling (no interpolation). Critical for the ray march's `Texture3D.Load()` which does exact integer-coordinate lookup.
- `wrapMode = TextureWrapMode.Clamp` = out-of-bounds coordinates clamp to the edge instead of wrapping. Prevents read artifacts at grid boundaries.

The normals texture uses `RenderTextureFormat.ARGBFloat` (4 × float32 = 16 bytes/voxel) and `FilterMode.Bilinear` for smooth interpolation when sampled between voxels.

Buffer Upload

```
void UploadDynamicTriangles()
{
    if (_dynamicTriBuffer != null && _dynamicTriBuffer.count < _dynamicTriCount)
    { _dynamicTriBuffer.Release(); _dynamicTriBuffer = null; }
    if (_dynamicTriBuffer == null)
        _dynamicTriBuffer = new ComputeBuffer(_dynamicTriCount,
Marshal.SizeOf(typeof(Tri)));
    _dynamicTriBuffer.SetData(_dynamicTriList);
}
```

The buffer is only reallocated if the new data is *larger* than the existing buffer (grow-only pattern). This avoids per-frame allocation when the dynamic triangle count is stable. `Marshal.SizeOf(typeof(Tri))` returns 48 (matching the GPU struct layout).

Unity API: `ComputeBuffer.SetData(List<T>)` — Copies data from CPU to GPU. This is the upload bottleneck for dynamic objects. For 10K triangles × 48 bytes = 480 KB, this is well within PCIe bandwidth limits.

Dispatch Helpers

```
void Dispatch3D(int kernel, int gx, int gy, int gz)
{
    coreCS.GetKernelThreadGroupSizes(kernel, out uint tx, out uint ty, out uint
```

```
tz);
    coreCS.Dispatch(kernel,
        Mathf.CeilToInt(gx / (float)tx),
        Mathf.CeilToInt(gy / (float)ty),
        Mathf.CeilToInt(gz / (float)tz));
}
```

Unity API: `ComputeShader.GetKernelThreadGroupSizes()` — Returns the `[numthreads(x,y,z)]` values declared in the HLSL kernel. These are compile-time constants baked into the shader. For a `[numthreads(8,8,8)]` kernel with a 100^3 grid, this dispatches `(13, 13, 13)` thread groups = 2197 groups × 512 threads = ~1.1M GPU threads.

Unity API: `ComputeShader.Dispatch(kernel, groupsX, groupsY, groupsZ)` — Queues the compute kernel for execution on the GPU. This is asynchronous — the CPU doesn't wait for the GPU to finish. Multiple sequential `Dispatch` calls pipeline naturally on the GPU command queue.

```
void DispatchSweep(int axis, int planeA, int planeB)
{
    coreCS.SetInt("_SweepAxis", axis);
    Dispatch2D(KSweepFill, planeA, planeB);
}
```

The sweep fill dispatches a 2D grid of threads. For sweep axis X, each thread covers one (Y, Z) line and sweeps along X. The dispatch dimensions are the perpendicular plane sizes.

Gizmos

```
void OnDrawGizmosSelected()
{
    if (boundsMode == BoundsMode.Manual)
    {
        Gizmos.color = new Color(1, 1, 0, 0.3f);
        Vector3 s = gridMax - gridMin;
        Gizmos.DrawWireCube(gridMin + s * 0.5f, s);
    }
```

Unity API: `OnDrawGizmosSelected()` — Called by the Unity editor when the GameObject is selected in the hierarchy. Draws wireframe visualization of the voxel grid bounds in the Scene view.

Key Algorithms / Design Patterns

1. **Static/Dynamic Split (Cache-and-Patch)** — A classic optimisation pattern where expensive computation results are cached and only the delta (moving objects) is recomputed per frame.
2. **Grow-Only Buffer Allocation** — Dynamic triangle buffers only grow, never shrink. Prevents allocation churn when triangle counts fluctuate slightly.

3. **Dirty Flag Pattern** — `_staticDirty` defers expensive work until the next frame update, preventing redundant rebuilds.
4. **Component-Based Scene Query** — Uses Unity's `FindObjectsByType` to discover all scene geometry. The `VoxelDynamic` marker component follows Unity's component pattern for runtime classification.

CPU / GPU Utilisation

Phase	Where	Work
Triangle extraction	CPU (`LateUpdate`)	`mesh.vertices` copy, `localToWorldMatrix` transform, normal computation
Triangle upload	CPU → GPU	`ComputeBuffer.SetData()` — DMA transfer
Static copy	GPU (linear kernel)	O(N) buffer copy, bandwidth-bound
Surface voxelisation	GPU (parallel per triangle)	SAT intersection, bandwidth + ALU
Flood fill	GPU (parallel per grid line)	Sequential along sweep direction, divergent branching
Texture build + blur + normals	GPU (parallel per voxel)	ALU-bound 3D passes

3. File: VoxelTracerCore.compute

General Purpose

This compute shader file contains **8 kernels** that collectively implement: surface voxelisation via SAT, axis-sweep interior flood fill, buffer-to-texture conversion, 3D box blur, gradient normal computation, and static cache management. It is the computational heart of the system.

HLSL Preamble & Declarations

```
#pragma kernel Clear
#pragma kernel Surface
#pragma kernel SweepFill
#pragma kernel BuildTexture
#pragma kernel ComputeNormals
#pragma kernel CopyStaticToWorking
#pragma kernel ClearFlood
#pragma kernel BlurFill
```

`#pragma kernel` directives tell Unity's shader compiler to compile each named function as a separate GPU kernel entry point. Each kernel gets its own dispatch and can bind different resources.

```
int _Width, _Height, _Depth;
float3 _Start;
```

```
float  _Unit, _HalfUnit;
```

Global uniforms set from C# via `ComputeShader.SetInt()` / `SetFloat()` / `SetVector()`. In HLSL compute shaders, these are stored in a constant buffer (cbuffer) automatically by Unity. All kernels in the same `.compute` file share these uniforms.

```
struct Tri { float3 a, b, c, normal; };
StructuredBuffer<Tri> _Tris;
```

`StructuredBuffer<T>` is a read-only GPU buffer. It maps 1:1 to the C# `ComputeBuffer` bound via `SetBuffer()`. Each `float3` in HLSL is 12 bytes (3 × float32), and the struct has 4 of them = 48 bytes — matching the C# `Tri` struct.

```
RWStructuredBuffer<uint> _VoxelBuffer;
RWStructuredBuffer<uint> _FloodBuffer;
```

`RWStructuredBuffer<uint>` is a read-write structured buffer. The `RW` prefix in HLSL stands for "Read-Write" and requires the buffer to be bound in unordered access view (UAV) mode. In Unity, any `ComputeBuffer` bound to an `RW` resource automatically uses UAV.

Each voxel is represented by a single `uint` — 0 means empty, non-zero means surface. This minimal representation (1 uint = 4 bytes) keeps memory low. At 128^3, the buffer is 8 MB.

Helper Functions

`VoxelIndex()`

```
int VoxelIndex(int x, int y, int z)
{
    return z * (_Width * _Height) + y * _Width + x;
}
```

Flattens 3D coordinates to a 1D buffer index. The memory layout is Z-major (z varies slowest, x varies fastest). This matches the standard C array-of-arrays layout `[z][y][x]`.

`VoxelCenter()`

```
float3 VoxelCenter(int x, int y, int z)
{
    return float3(
        _Start.x + _Unit * x + _HalfUnit,
        _Start.y + _Unit * y + _HalfUnit,
        _Start.z + _Unit * z + _HalfUnit
```

```
    );
}
```

Returns the world-space center of voxel `(x,y,z)`. The voxel at `(0,0,0)` has its minimum corner at `_Start` and its center at `_Start + (_HalfUnit, _HalfUnit, _HalfUnit)`.

`AxisOverlap()` — SAT Axis Test

```
bool AxisOverlap(float3 v0, float3 v1, float3 v2, float3 ext, float3 axis)
{
    float p0 = dot(v0, axis);
    float p1 = dot(v1, axis);
    float p2 = dot(v2, axis);
    float r  = ext.x * abs(axis.x) + ext.y * abs(axis.y) + ext.z * abs(axis.z);
    return !((max(p0, max(p1, p2)) < -r) || (r < min(p0, min(p1, p2))));
}
```

Projects the three triangle vertices and the AABB half-extents onto a candidate separating axis. If the maximum triangle projection is less than `-r` (AABB radius on this axis) OR the minimum is greater than `+r`, the projections don't overlap on this axis, meaning the shapes are separated.

The operation `ext.x * abs(axis.x) + ext.y * abs(axis.y) + ext.z * abs(axis.z)` computes the AABB's **projected radius** onto the axis. This works because an AABB's projection onto any axis is the sum of each half-extent scaled by the absolute component of the axis in that dimension.

Note: Both vertices and AABB are centred at the origin (the triangle vertices `v0/v1/v2` are pre-translated by subtracting the AABB centre). This simplifies the AABB projection to `[-r, +r]`.

`TriAABBIntersect()` — Full SAT Triangle-AABB Test

```
bool TriAABBIntersect(float3 va, float3 vb, float3 vc, float3 center, float3 ext)
{
    float3 v0 = va - center;
    float3 v1 = vb - center;
    float3 v2 = vc - center;

    float3 f0 = v1 - v0;  // edge 0
    float3 f1 = v2 - v1;  // edge 1
    float3 f2 = v0 - v2;  // edge 2
```

This is the **Tomas Akenine-Möller** triangle-box intersection test (originally published 2001, widely cited). It tests 13 separating axes:

9 cross-product axes (AABB edges × triangle edges):

```
    // AABB X-axis × each triangle edge
    if (!AxisOverlap(v0, v1, v2, ext, float3(0, -f0.z, f0.y))) return false;
    if (!AxisOverlap(v0, v1, v2, ext, float3(0, -f1.z, f1.y))) return false;
    if (!AxisOverlap(v0, v1, v2, ext, float3(0, -f2.z, f2.y))) return false;
    // AABB Y-axis × each triangle edge
    if (!AxisOverlap(v0, v1, v2, ext, float3(f0.z, 0, -f0.x))) return false;
    // ...
    // AABB Z-axis × each triangle edge
    if (!AxisOverlap(v0, v1, v2, ext, float3(-f0.y, f0.x, 0))) return false;
    // ...
```

The cross products are pre-expanded analytically. For example, `AABB_X × f0 = (1,0,0) × (f0.x, f0.y, f0.z) = (0, -f0.z, f0.y)`. This avoids calling a general `cross()` function.

3 AABB face normals (axis-aligned):

```
    if (max(v0.x, max(v1.x, v2.x)) < -ext.x || min(v0.x, min(v1.x, v2.x)) > ext.x)
return false;
    if (max(v0.y, max(v1.y, v2.y)) < -ext.y || min(v0.y, min(v1.y, v2.y)) > ext.y)
return false;
    if (max(v0.z, max(v1.z, v2.z)) < -ext.z || min(v0.z, min(v1.z, v2.z)) > ext.z)
return false;
```

These test whether the triangle is entirely on one side of any AABB face. Since the AABB is centred at the origin, this simplifies to comparing vertex extremes against the half-extents.

1 triangle face normal:

```
    float3 n = cross(f0, f1);
    // ...
    float d = dot(n, va);
    float r = ext.x * abs(n.x) + ext.y * abs(n.y) + ext.z * abs(n.z);
    float s = dot(n, center) - d;
    return abs(s) <= r;
```

Tests whether the AABB lies entirely on one side of the triangle's plane.

Total: 9 + 3 + 1 = 13 axes. If ALL axes show overlap, the triangle and AABB intersect. If ANY axis shows separation, they don't.

Kernel Walkthroughs

Kernel 0: `Clear`

```
[numthreads(8, 8, 8)]
void Clear(uint3 id : SV_DispatchThreadID)
```

```
{
    if ((int)id.x >= _Width || (int)id.y >= _Height || (int)id.z >= _Depth)
return;
    int idx = VoxelIndex((int)id.x, (int)id.y, (int)id.z);
    _VoxelBuffer[idx] = 0u;
    _FloodBuffer[idx] = 0u;
    _FillTex[id] = 0.0;
    _NormalTex[id] = float4(0, 0, 0, 0);
}
```

`SV_DispatchThreadID` — A system-value semantic in HLSL that gives each thread its global 3D index across all thread groups. For thread group (1, 2, 3) with `[numthreads(8,8,8)]`, `SV_DispatchThreadID` for the first thread is `(8, 16, 24)`.

Bounds check: `if ((int)id.x >= _Width ...)` — Since dispatch dimensions are rounded up (`CeilToInt`), there may be extra threads beyond the grid boundary. These threads must bail out to avoid out-of-bounds writes.

`[numthreads(8, 8, 8)]` — Declares 512 threads per thread group. For 3D spatial kernels, 8^3 is a standard choice that matches the hardware's warp/wavefront size (32 on NVIDIA, 64 on AMD) while providing good spatial locality.

Kernel 1: `Surface` — SAT Surface Voxelisation

```
[numthreads(64, 1, 1)]
void Surface(uint3 id : SV_DispatchThreadID)
{
    if (id.x >= (uint)_TriCount) return;
    Tri tri = _Tris[id.x];
```

Each GPU thread processes one triangle. `[numthreads(64,1,1)]` uses a 1D thread group because the work is linear (one thread per triangle), not spatial. 64 is chosen for good occupancy.

```
    float3 triMin = min(tri.a, min(tri.b, tri.c));
    float3 triMax = max(tri.a, max(tri.b, tri.c));
    int3 gmin = int3(floor((triMin - _Start) / _Unit)) - 1;
    int3 gmax = int3(floor((triMax - _Start) / _Unit)) + 1;
    gmin = clamp(gmin, 0, int3(_Width - 1, _Height - 1, _Depth - 1));
    gmax = clamp(gmax, 0, int3(_Width - 1, _Height - 1, _Depth - 1));
```

Computes the triangle's axis-aligned bounding box in voxel coordinates, expanded by 1 voxel in each direction (the `-1` / `+1`). This ensures conservative coverage — a triangle touching a voxel corner won't be missed.

```
    for (int z = gmin.z; z <= gmax.z; z++)
    for (int y = gmin.y; y <= gmax.y; y++)
```

```
    for (int x = gmin.x; x <= gmax.x; x++)
    {
        float3 center = VoxelCenter(x, y, z);
        if (TriAABBIntersect(tri.a, tri.b, tri.c, center, ext))
        {
            uint dummy;
            InterlockedOr(_VoxelBuffer[VoxelIndex(x, y, z)], 1u, dummy);
        }
    }
```

The nested loop iterates over the triangle's local AABB and tests each voxel with the full 13-axis SAT.

`InterlockedOr` — An HLSL atomic intrinsic that performs a bitwise OR on a buffer element atomically. Multiple threads can safely write to the same voxel simultaneously (when different triangles overlap the same voxel). The `dummy` output variable receives the previous value (unused here but required by the function signature).

Why `InterlockedOr` instead of a simple write: Without atomics, two threads writing to the same voxel would race. If thread A sets voxel to 1 and thread B sets it to 1 later, a simple write works by coincidence. But if the representation were richer (e.g., storing triangle indices or normals), the race would corrupt data. Using atomics is defensive and correct regardless of future changes.

Kernel 2: `SweepFill` — Axis-Sweep Flood Fill

```
[numthreads(8, 8, 1)]
void SweepFill(uint3 id : SV_DispatchThreadID)
{
    int a = (int)id.x;
    int b = (int)id.y;
    // ...
    // Forward sweep
    bool propagating = true;
    for (int s = 0; s < dimSweep; s++)
    {
        int idx = VoxelIndex(x, y, z);
        if (_VoxelBuffer[idx] > 0u)
        {
            propagating = false;
            continue;
        }
        if (propagating || _FloodBuffer[idx] > 0u)
        {
            _FloodBuffer[idx] = 1u;
            propagating = true;
        }
        else
        {
            propagating = false;
        }
    }
```

This implements **bidirectional outside-flood propagation** along one axis. Each thread handles one line through the grid.

Algorithm:

1. Start from the boundary face with `propagating = true` (boundary voxels are "outside").
2. Walk along the line. If the current voxel is a surface (`_VoxelBuffer > 0`), stop propagating.
3. If propagating OR the voxel was already marked as outside by a prior sweep axis (`_FloodBuffer > 0`), mark it as outside and continue propagating.
4. If reaching a non-surface, non-outside voxel while not propagating, leave it unmarked (it's a potential interior voxel).
5. Repeat backward from the opposite boundary.

Key insight: The `|| _FloodBuffer[idx] > 0u` check enables cross-axis propagation. A voxel marked "outside" by the X sweep can restart propagation during the Y sweep. This handles L-shaped exterior regions that no single axis can reach.

`[numthreads(8, 8, 1)]` — Each thread group covers an 8×8 patch of the perpendicular plane. The `1` in Z means each thread handles one full line (the sequential sweep loop).

Limitation: This is not a true flood fill — it can't handle very complex concavities where the exterior wraps around multiple times. The `fillSweepRounds` parameter helps by repeating the 3-axis sweep, allowing information to propagate further.

Kernel 3: `BuildTexture`

```
[numthreads(8, 8, 8)]
void BuildTexture(uint3 id : SV_DispatchThreadID)
{
    uint surfVal = _VoxelBuffer[idx];
    bool isSurface = surfVal > 0u;

    if (_FillVolume == 0)
    {
        _FillTex[id] = isSurface ? 1.0 : 0.0;
    }
    else
    {
        bool isOutside = _FloodBuffer[idx] > 0u;
        _FillTex[id] = (!isOutside || isSurface) ? 1.0 : 0.0;
    }
}
```

Converts the dual-buffer representation (surface + flood) into a single 3D texture:

- **Surface only mode** (`_FillVolume == 0`): only surface voxels are filled.
- **Volume mode**: a voxel is filled if it's either a surface voxel OR not marked as outside. This gives the union of surfaces and interior.

The expression `!isOutside || isSurface` is logically equivalent to: "filled = interior ∪ surface". Interior voxels are defined as `!surface && !outside`.

Kernel 4: `ComputeNormals` — Gradient-Based Normal Estimation

```
[numthreads(8, 8, 8)]
void ComputeNormals(uint3 id : SV_DispatchThreadID)
{
    float here = _FillTex[p];
    if (here < 0.5)
    {
        _NormalTex[p] = float4(0, 0, 0, 0);
        return;
    }

    float gx = _BlurredFillTex[int3(min(p.x + 1, _Width  - 1), p.y, p.z)]
             - _BlurredFillTex[int3(max(p.x - 1, 0),           p.y, p.z)];
    // ... gy, gz similarly ...
    float3 n = -grad * rsqrt(len2);
    _NormalTex[p] = float4(n, 1);
}
```

Computes the **central-difference gradient** of the blurred fill field:

- `gx = blurred[x+1] - blurred[x-1]` (similarly for Y, Z)
- The gradient of a binary "inside/outside" field points from filled towards empty.
- The outward surface normal is the **negated** normalised gradient (`-grad`).

`rsqrt(len2)` — HLSL intrinsic for reciprocal square root (`1.0 / sqrt(x)`). Hardware-accelerated on GPUs (single clock cycle on most architectures). More efficient than `normalize()` when you also need to check for zero-length.

Why the normals read from `_BlurredFillTex` but the fill check reads `_FillTex`: The original binary fill field has infinitely sharp edges — its gradient is a Dirac delta function at surfaces and zero everywhere else. The blurred version has smooth transitions, yielding meaningful gradients at surface voxels. But the fill check still uses the un-blurred texture to avoid marking partially-filled voxels as "no normal".

Boundary handling: `min(p.x + 1, _Width - 1)` and `max(p.x - 1, 0)` clamp neighbour lookups to valid coordinates. At boundaries, this effectively duplicates the edge value, making the gradient zero component in that direction.

Kernel 5: `CopyStaticToWorking`

```
[numthreads(256, 1, 1)]
void CopyStaticToWorking(uint3 id : SV_DispatchThreadID)
{
    int idx = (int)id.x;
    if (idx >= _TotalVoxels) return;
```

```
    _VoxelBuffer[idx] = _StaticVoxelBuffer[idx];
}
```

Pure linear copy. `[numthreads(256,1,1)]` maximises occupancy for linear work — 256 threads per group, 1D dispatch. For 2M voxels, this is ~8K thread groups, completing in microseconds (bandwidth-bound, ~8 MB copy).

Kernel 6: `ClearFlood`

Identical structure to `CopyStaticToWorking`, but writes zeros. Separated from `Clear` because the per-frame path doesn't need to clear `_VoxelBuffer` (it's overwritten by `CopyStaticToWorking`) or textures (they're overwritten by `BuildTexture` and `ComputeNormals`).

Kernel 7: `BlurFill`

```
[numthreads(8, 8, 8)]
void BlurFill(uint3 id : SV_DispatchThreadID)
{
    float sum = 0.0;
    for (int dz = -1; dz <= 1; dz++)
    for (int dy = -1; dy <= 1; dy++)
    for (int dx = -1; dx <= 1; dx++)
    {
        int3 q = clamp(p + int3(dx, dy, dz), 0, int3(_Width - 1, _Height - 1,
_Depth - 1));
        sum += _FillTex[q];
    }
    _BlurredFillTex[p] = sum / 27.0;
}
```

A **3×3×3 box blur** — averages the 27 voxels in the local cubic neighbourhood. For a binary fill field (values 0 or 1), the blurred output takes values in {0/27, 1/27, ..., 27/27}. At a surface boundary where roughly half the neighbourhood is filled, the value is ~0.5, creating a smooth transition zone.

Why 27: 3^3 = 27 samples in a 3×3×3 cube. The `/ 27.0` normalises the sum so a fully-filled neighbourhood gives 1.0 and fully-empty gives 0.0.

`clamp(p + int3(dx, dy, dz), ...)` — Clamps neighbour coordinates to valid ranges. At grid boundaries, this replicates the edge value (border extrapolation). This is equivalent to the `Clamp` addressing mode in texture samplers.

Key Algorithms / Design Patterns

1. **Separating Axis Theorem (SAT)** — The standard mathematical test for convex polytope intersection. For AABB vs triangle, exactly 13 axes suffice for a complete test.
2. **Sweep-based flood fill** — Approximates a true BFS/DFS flood fill using independent line sweeps. Parallelises well on GPUs (each line is independent).

3. **Ping-free normal estimation** — Uses blur→gradient instead of storing per-triangle normals. Avoids the race condition problem of writing normals from overlapping triangles.
4. **Atomic writes for concurrent voxelisation** — Multiple threads safely mark the same voxel using `InterlockedOr`.

CPU vs GPU Utilisation

All 8 kernels execute entirely on the GPU. The CPU's role is limited to:

- Setting uniform parameters via `SetInt/SetFloat/SetVector/SetBuffer/SetTexture`
- Issuing `Dispatch()` calls (which are non-blocking GPU command submissions)
- The one-time static cache readback via `GetData()` (GPU → CPU, blocking)

---

4. File: VoxelTracerRayMarch.compute

General Purpose

This compute shader implements a **full-screen DDA ray march** that renders the voxel volume to a 2D render target. Each pixel casts a ray through the 3D voxel grid, finds the first filled voxel, computes shading, and writes the result. It runs once per frame, dispatched by `VoxelTracerCamera`.

Function Walkthrough

Uniforms

```
float4x4 _CamToWorld;    // Camera.cameraToWorldMatrix
float4x4 _InvProj;       // Camera.projectionMatrix.inverse
float3   _CamPos;        // Camera position in world space
float2   _ScreenSize;    // (width, height) in pixels
```

These are set from `VoxelTracerCamera.cs`. The two matrices together allow reconstructing a world-space ray direction from pixel coordinates.

`RayAABB()` — Ray-Box Intersection

```
float2 RayAABB(float3 ro, float3 rd, float3 bmin, float3 bmax)
{
    float3 inv = 1.0 / rd;
    float3 t0 = (bmin - ro) * inv;
    float3 t1 = (bmax - ro) * inv;
    float3 mn = min(t0, t1);
    float3 mx = max(t0, t1);
    return float2(max(mn.x, max(mn.y, mn.z)), min(mx.x, min(mx.y, mx.z)));
}
```

The **slab method** for ray-AABB intersection. For each axis, the ray enters the slab at `t = (bmin - ro) / rd` and exits at `t = (bmax - ro) / rd` (or vice versa if `rd` is negative — hence the `min/max` swap). The ray is inside the box when it's inside ALL three slabs simultaneously, which gives the intersection range `[max of entry t's, min of exit t's]`. If `tEntry > tExit`, there's no intersection.

Ray Construction

```
float2 uv  = (float2(id.xy) + 0.5) / _ScreenSize;
float2 ndc = uv * 2.0 - 1.0;
float4 viewPos = mul(_InvProj, float4(ndc, -1.0, 1.0));
viewPos.xyz /= viewPos.w;
float3 rd = normalize(mul((float3x3)_CamToWorld, viewPos.xyz));
```

1. `uv` maps pixel centre to [0,1] range (the `+0.5` centres within the pixel).
2. `ndc` maps to [-1,1] normalised device coordinates.
3. `mul(_InvProj, ...)` unprojects from clip space to view space. The `-1.0` Z means the near plane in OpenGL convention (Unity uses OpenGL convention for projection matrices).
4. `viewPos.xyz /= viewPos.w` — perspective divide to get the actual view-space direction.
5. `mul((float3x3)_CamToWorld, ...)` rotates the view-space direction to world space. Casting to `float3x3` discards the translation component (we only want direction, not position).

```
rd.x += (abs(rd.x) < 1e-8) ? 1e-8 : 0.0;
```

Zero-component nudge: If a ray direction component is exactly zero, `1.0 / rd.x` would be infinity, causing NaN propagation in the DDA. Adding a tiny epsilon avoids this while imperceptibly altering the ray direction.

DDA Setup

```
float tStart = max(tRange.x + _Unit * 0.001, 0.0);
float3 entry = ro + rd * tStart;
float3 gridF = (entry - _Start) / _Unit;

int3 voxel = int3(floor(gridF));
voxel = clamp(voxel, 0, int3(_Width - 1, _Height - 1, _Depth - 1));
```

Advances the ray to just past the AABB entry point (the `+ _Unit * 0.001` prevents the first sample from landing exactly on the boundary, which could cause floating-point ambiguity).

`gridF` is the entry point in continuous grid coordinates. `floor()` gives the integer voxel index.

```
int3 stepDir = int3(
    rd.x >= 0.0 ? 1 : -1,
    rd.y >= 0.0 ? 1 : -1,
```

```
    rd.z >= 0.0 ? 1 : -1
);
```

The step direction per axis — which way the ray moves through the grid on each axis.

```
float3 invRd  = 1.0 / rd;
float3 tDelta = abs(invRd) * _Unit;
```

`tDelta` is the parametric distance along the ray to cross one voxel on each axis. For a horizontal ray (`rd = (1,0,0)`), `tDelta.x = _Unit` and `tDelta.y = tDelta.z = infinity`.

```
float3 nextBound;
nextBound.x = _Start.x + ((stepDir.x > 0) ? (voxel.x + 1) : voxel.x) * _Unit;
float3 tMax = (nextBound - ro) * invRd;
```

`tMax` is the parametric distance to the NEXT voxel boundary on each axis. This is the key DDA state variable — whichever axis has the smallest `tMax` is the one that gets stepped next.

DDA Traversal Loop

```
bool hit = false;
int lastAxis = -1;

for (int i = 0; i < _MaxSteps; i++)
{
    if (any(voxel < 0) || any(voxel >= int3(_Width, _Height, _Depth)))
        break;

    float fill = _FillTex.Load(int4(voxel, 0));
    if (fill > 0.5)
    {
        hit = true;
        break;
    }

    if (tMax.x < tMax.y)
    {
        if (tMax.x < tMax.z)
        { voxel.x += stepDir.x; tMax.x += tDelta.x; lastAxis = 0; }
        else
        { voxel.z += stepDir.z; tMax.z += tDelta.z; lastAxis = 2; }
    }
    else
    {
        if (tMax.y < tMax.z)
        { voxel.y += stepDir.y; tMax.y += tDelta.y; lastAxis = 1; }
```

```
        else
        { voxel.z += stepDir.z; tMax.z += tDelta.z; lastAxis = 2; }
    }
}
```

This is the **Amanatides-Woo DDA algorithm** (1987). At each step:

1. Check if the current voxel is filled. If yes, we have a hit.
2. Find which axis boundary is closest (smallest `tMax`).
3. Step the voxel index in that direction, advance `tMax` by one voxel width.
4. Record `lastAxis` for normal computation.

`_FillTex.Load(int4(voxel, 0))` — `Texture3D.Load()` is equivalent to `texelFetch()` in GLSL. It reads the exact texel at integer coordinates without any filtering. The 4th component `0` is the mip level. This is preferred over `_FillTex[voxel]` because `Load()` is defined by the HLSL specification for `Texture3D` (non-RW) resources, while array indexing (`[]`) works for `RWTexture3D`.

`any(voxel < 0)` — HLSL intrinsic that returns `true` if any component of the boolean vector is true. This is a boundary check — if the ray exits the grid, we stop.

Normal from DDA Face Crossing

```
float3 normal;
if (lastAxis == 0)
    normal = float3((float)(-stepDir.x), 0.0, 0.0);
else if (lastAxis == 1)
    normal = float3(0.0, (float)(-stepDir.y), 0.0);
else if (lastAxis == 2)
    normal = float3(0.0, 0.0, (float)(-stepDir.z));
```

The normal is the face of the voxel the ray entered through, pointing back toward the camera. If the last DDA step was on the X axis with `stepDir.x = +1` (ray moving in +X), the ray entered through the -X face, so the normal is `(-1, 0, 0)`.

Edge case — first voxel hit without any DDA step:

```
else
{
    float3 absRd = abs(rd);
    if (absRd.x > absRd.y && absRd.x > absRd.z)
        normal = float3(rd.x > 0.0 ? -1.0 : 1.0, 0.0, 0.0);
    // ...
}
```

If the camera is inside a voxel (or the entry point is already filled), `lastAxis` is still -1. The fallback uses the dominant ray direction to guess which face was entered.

Shading

```
if (_VisMode == 1)
{
    color = normal * 0.5 + 0.5;  // map [-1,1] → [0,1] for visualisation
}
else
{
    float3 L = normalize(_LightDir);
    float NdotL = max(0.0, dot(normal, L));
    color = _SurfaceColor * _LightColor * NdotL + _AmbientColor;
}
```

Lit mode: Simple Lambertian diffuse shading: `diffuse = albedo × light × max(0, N·L) + ambient`. No specular, no shadows, no global illumination. The 6-direction DDA normals give a clean Minecraft-style flat-shaded look.

Normals mode: The `* 0.5 + 0.5` remapping is standard for normal visualisation — it maps (-1,-1,-1) to (0,0,0) and (+1,+1,+1) to (1,1,1). An X-facing normal appears red, Y-facing green, Z-facing blue.

Key Algorithms / Design Patterns

1. **Amanatides-Woo 3D DDA** — The standard grid traversal algorithm used in voxel engines since Wolfenstein 3D. O(N) where N is the number of voxels traversed (typically 50-500 per ray).
2. **Face-crossing normals** — Derives geometry normals directly from the traversal state, requiring zero memory for normal storage.
3. **Ray-AABB slab method** — The standard O(1) ray-box intersection test.

CPU vs GPU Utilisation

This kernel is 100% GPU. Each pixel is an independent thread — perfect GPU parallelism. For 1920×1080, it dispatches `(240, 135, 1)` thread groups = 32,400 groups × 64 threads = ~2M threads. The main bottleneck is texture fetch latency (random access pattern as rays traverse different grid regions).

---

5. File: VoxelTracerCamera.cs

General Purpose

Camera component that dispatches the DDA ray march compute shader and composites the result over Unity's normal scene rendering. It acts as the bridge between the voxelisation system and the screen.

Function Walkthrough

Class Declarations

```
[RequireComponent(typeof(Camera))]
public sealed class VoxelTracerCamera : MonoBehaviour
```

Unity API: `[RequireComponent(typeof(Camera))]` — Unity attribute that automatically adds a `Camera` component to the GameObject if one doesn't exist, and prevents accidental removal. This is used because the script relies on `GetComponent<Camera>()` and the `OnRenderImage` callback (which only fires on cameras).

```
static readonly int _SrcTex = Shader.PropertyToID("_SrcTex");
static readonly int _VoxTex = Shader.PropertyToID("_VoxTex");
```

Unity API: `Shader.PropertyToID()` — Converts a shader property name string to an integer ID for faster property lookups. Called once (static readonly), the integer is reused every frame instead of doing string hashing.

`OnEnable()`

```
_kernel = rayMarchCS.FindKernel("RayMarch");
_compositeMat = new Material(Shader.Find("Hidden/VoxelComposite"));
```

Unity API: `ComputeShader.FindKernel(name)` — Returns the integer index of the named kernel within the compute shader. Indices are assigned in declaration order (0-based).

Unity API: `Shader.Find("Hidden/VoxelComposite")` — Searches all loaded shaders for one with the matching name. The `Hidden/` prefix means it won't appear in Unity's material shader dropdown. `new Material(shader)` creates a runtime material instance.

`OnRenderImage()` **— The Render Pipeline Hook**

```
void OnRenderImage(RenderTexture src, RenderTexture dest)
```

Unity API: `OnRenderImage(src, dest)` — Built-in render pipeline callback. Called after the camera finishes rendering but before the result is displayed. `src` contains the camera's rendered scene, `dest` is the final output. You MUST either `Graphics.Blit(src, dest)` or write to `dest` — if you don't, the screen may show garbage.

Important: This callback only exists in the built-in render pipeline. In URP/HDRP, you would use `ScriptableRenderPass` instead.

```
rayMarchCS.SetMatrix("_CamToWorld", _cam.cameraToWorldMatrix);
rayMarchCS.SetMatrix("_InvProj", _cam.projectionMatrix.inverse);
```

Unity API: `Camera.cameraToWorldMatrix` — The inverse of the view matrix. Transforms from camera (view) space to world space. Used by the ray march to convert screen pixels to world-space ray directions.

Unity API: `Camera.projectionMatrix` — The projection matrix used by the GPU. Its `.inverse` maps clip/NDC coordinates back to view space.

```
rayMarchCS.Dispatch(_kernel, Mathf.CeilToInt(w / 8f), Mathf.CeilToInt(h / 8f), 1);
```

Dispatches the ray march with one thread per pixel. Thread groups are 8×8 (matching `[numthreads(8,8,1)]`), so a 1920×1080 screen dispatches 240×135 groups.

Compositing

```
_compositeMat.SetTexture(_VoxTex, _colorRT);
Graphics.Blit(src, dest, _compositeMat);
```

Unity API: `Graphics.Blit(source, dest, material)` — Draws a full-screen quad with the given material. The material's `_MainTex` is automatically set to `source`. The fragment shader then blends `source` (scene) with `_VoxTex` (voxel render) using alpha-over compositing.

Render Target Management

```
_colorRT = new RenderTexture(w, h, 0, RenderTextureFormat.ARGBFloat)
{
    enableRandomWrite = true,
    filterMode = FilterMode.Point,
    useMipMap = false
};
_colorRT.Create();
```

`ARGBFloat` = 128 bits/pixel (4 × float32). `enableRandomWrite = true` is required because the compute shader writes to it as `RWTexture2D<float4>`. `FilterMode.Point` prevents bilinear blurring when sampled in the composite shader.

Key Algorithms / Design Patterns

1. **Post-processing compositing** — Uses Unity's `OnRenderImage` callback to overlay voxel rendering on top of the normal scene.
2. **Lazy RT allocation** — Render targets are only (re)allocated when the screen resolution changes.

CPU vs GPU Utilisation

- **CPU:** Sets ~20 shader uniforms, dispatches 1 compute kernel, performs 1 blit. Negligible CPU cost.
- **GPU:** The ray march kernel is the rendering bottleneck. At 1080p with 1024 max steps, each pixel may test up to 1024 voxels with texture lookups. This is the most GPU-intensive operation in the whole system.

---

6. File: VoxelComposite.shader

General Purpose

A simple full-screen post-processing shader that composites the voxel render over the scene using alpha blending.

Code Walkthrough

```
Shader "Hidden/VoxelComposite"
```

The `Hidden/` prefix hides this shader from Unity's shader selection dropdown in the Inspector.

```
Cull Off ZWrite Off ZTest Always
```

Standard post-processing render state:

- `Cull Off` — Don't cull any faces (the full-screen quad might have any winding).
- `ZWrite Off` — Don't write to the depth buffer (post-processing shouldn't affect depth).
- `ZTest Always` — Ignore the depth buffer entirely (always draw).

```
v2f vert (appdata v)
{
    v2f o;
    o.vertex = UnityObjectToClipPos(v.vertex);
    o.uv = v.uv;
    return o;
}
```

Unity API: `UnityObjectToClipPos()` — Built-in Unity shader function (from `UnityCG.cginc`) that transforms a vertex from object space to clip space. Equivalent to `mul(UNITY_MATRIX_MVP, vertex)`.

```
fixed4 frag (v2f i) : SV_Target
{
    fixed4 scene = tex2D(_MainTex, i.uv);
    fixed4 voxel = tex2D(_VoxTex, i.uv);
    return lerp(scene, fixed4(voxel.rgb, 1.0), voxel.a);
}
```

Alpha-over compositing: Where the voxel alpha is 1.0 (ray hit), the output shows the voxel colour. Where alpha is 0.0 (ray miss/transparent), the output shows the scene. `lerp(a, b, t) = a*(1-t) + b*t`.

`fixed4` — Low-precision (11-bit) float type in Unity shaders. Sufficient for final colour output. On modern GPUs, this compiles to `half` or `float` depending on the platform.

Key Algorithms / Design Patterns

- **Alpha-over blend** — Standard compositing operation: `result = foreground * alpha + background * (1 - alpha)`.

---

7. File: VoxelTracerComposite.shader

General Purpose

A more advanced compositor that uses **depth comparison** to correctly occlude voxels behind scene geometry.

Code Walkthrough

```
sampler2D _CameraDepthTexture;
```

Unity API: `_CameraDepthTexture` — Unity automatically populates this global sampler with the camera's depth buffer when `Camera.depthTextureMode` includes `DepthTextureMode.Depth`. Values are non-linear (perspective-corrected Z buffer values).

```
float rawDepth  = SAMPLE_DEPTH_TEXTURE(_CameraDepthTexture, i.uv);
float sceneDepth = LinearEyeDepth(rawDepth);
```

Unity API: `SAMPLE_DEPTH_TEXTURE()` — Macro from `UnityCG.cginc` that handles platform differences in depth texture sampling (e.g., DX11 vs OpenGL reversed-Z).

Unity API: `LinearEyeDepth()` — Converts the non-linear depth buffer value to linear eye-space distance (in world units). Unity's depth buffer stores `1/z` values for precision, so this function performs `1.0 / (_ZBufferParams.z * depth + _ZBufferParams.w)`.

```
if (vDepth > 0.0 && vDepth < 1e9 && vDepth < sceneDepth)
    return voxel;
return scene;
```

If the voxel hit is closer to the camera than the scene geometry at that pixel, show the voxel. Otherwise, show the scene. This prevents voxels from rendering on top of scene objects that are actually in front of them.

Note: This shader requires the ray march to output depth, which the current `VoxelTracerRayMarch.compute` does NOT do (it only outputs colour). This shader appears to be left over from a previous version or intended for future use.

---

8. File: VoxelDynamic.cs

General Purpose

A **marker component** — an empty MonoBehaviour whose sole purpose is to tag GameObjects as "dynamic" for the voxeliser.

```
public sealed class VoxelDynamic : MonoBehaviour { }
```

`sealed` prevents inheritance (minor performance optimisation for Unity's `GetComponent` queries). The class has no fields, no methods — its presence on a GameObject is the entire signal.

Design pattern: This is the **Marker/Tag Component** pattern, common in Unity. Other approaches (layers, tags) exist but components are more discoverable and type-safe — you can query for them with `FindObjectsByType<VoxelDynamic>()`.

---

9. File: VoxelNormalGizmos.cs

General Purpose

Debug visualisation tool that reads voxel data back from GPU to CPU and draws normal lines on surface voxels using GL immediate-mode rendering.

Function Walkthrough

```
[DefaultExecutionOrder(100)]
```

Unity API: `[DefaultExecutionOrder(N)]` — Controls the order of `Update()`, `LateUpdate()`, and `OnRenderImage()` callbacks among scripts. Higher values execute later. `100` ensures this runs after `VoxelTracerCamera` (default order `0`), so the voxel composite is already in the image before lines are drawn on top.

GPU Readback: `RebuildLines()`

```
Graphics.CopyTexture(fillRT, z, 0, tempRT, 0, 0);
var prev = RenderTexture.active;
RenderTexture.active = tempRT;
tempTex.ReadPixels(new Rect(0, 0, nx, ny), 0, 0, false);
tempTex.Apply(false);
RenderTexture.active = prev;
var raw = tempTex.GetRawTextureData<float>();
```

This is a per-slice GPU→CPU readback:

1. `Graphics.CopyTexture(src, srcSlice, srcMip, dst, dstSlice, dstMip)` — GPU-side copy of one slice (Z layer) of the 3D texture to a 2D render texture. This is a fast GPU→GPU operation.

2. `RenderTexture.active = tempRT` — Sets the active render target. `ReadPixels` reads from whatever `RenderTexture.active` is.

3. `Texture2D.ReadPixels(rect, destX, destY, recalcMip)` — Reads pixel data from the active render texture into a `Texture2D` on the CPU. This is a GPU→CPU readback and causes a pipeline stall (the CPU waits for the GPU to finish all pending work).

4. `GetRawTextureData<float>()` — Returns the raw byte data as a `NativeArray<float>`, allowing direct memory access without intermediate copies.

Performance note: This readback is expensive (stalls the GPU pipeline) and runs every `refreshInterval` seconds, not every frame. This is why the default `refreshInterval` is 0.2 seconds.

GL Immediate Mode Drawing

```
_glMat.SetPass(0);
GL.Begin(GL.LINES);
GL.Color(normalColor);
for (int i = 0; i < _lineCount; i++)
{
    GL.Vertex(_lineStarts[i]);
    GL.Vertex(_lineEnds[i]);
}
GL.End();
```

Unity API: GL immediate mode — Unity's wrapper over OpenGL-style immediate-mode rendering. `GL.Begin(GL.LINES)` starts a line primitive batch. Each pair of `GL.Vertex()` calls defines one line segment. `GL.End()` submits the batch. The `SetPass(0)` activates the material's first shader pass.

`GL.LoadProjectionMatrix(GL.GetGPUProjectionMatrix(...))` — Corrects the projection matrix for the current GPU API (OpenGL vs DirectX have different clip-space conventions). `GL.GetGPUProjectionMatrix` handles the reversal of the Y axis and the Z range ([-1,1] vs [0,1]).

Key Algorithms / Design Patterns

1. **GPU readback with throttling** — Reads voxel data at a configurable rate to avoid performance spikes.
2. **Surface detection via 6-connectivity** — A filled voxel is "surface" if at least one of its 6 face-adjacent neighbours is empty.
3. **Central-difference gradient normals** — Same algorithm as the GPU `ComputeNormals` kernel, reimplemented on CPU for the debug gizmo.

---

10. File: VoxelSliceViewer.cs

General Purpose

A debug overlay that displays a 2D cross-section (slice) of the 3D voxel volume on screen. Useful for verifying that surface voxelisation, flood fill, and volume fill are correct.

Function Walkthrough

```
[DefaultExecutionOrder(200)]
```

Even later than `VoxelNormalGizmos` (100), ensuring it runs last in the rendering chain.

Controls

```
if (Input.GetKeyDown(toggleKey))   // F2 toggles the overlay
    _visible = !_visible;
float scroll = Input.GetAxis("Mouse ScrollWheel");  // scrubs slice position
if (Input.GetKeyDown(KeyCode.Alpha1)) axis = SliceAxis.X;  // axis switching
```

Standard Unity input polling. `Input.GetKeyDown` returns true on the frame the key is pressed.

Slice Creation: `BuildSliceTexture()`

Reads the 3D fill data from GPU (same per-slice technique as `VoxelNormalGizmos`), then creates a `Texture2D` with coloured pixels:

```
if (fill > 0.5f)
{
    if (highlightSurface && IsSurface(x, y, z))
        c = surfaceColor;    // green for surface voxels
    else
        c = filledColor;     // white for interior
}
else
{
    c = emptyColor;           // dark for empty
}
```

Surface detection uses the same 6-connectivity test as `VoxelNormalGizmos`.

OnGUI Display

```
void OnGUI()
{
    GUI.DrawTexture(new Rect(imgX, imgY, imgW, imgH), _sliceTex,
ScaleMode.StretchToFill, true);
```

Unity API: `OnGUI()` — Legacy immediate-mode GUI callback. Called every frame when GUI events are processed. `GUI.DrawTexture()` renders a texture in screen space. Despite being "legacy", it's the simplest

way to draw arbitrary textures as UI overlays without setting up a Canvas.

The overlay is rendered with a dark semi-transparent background (`Color(0,0,0,0.7f)`) and a green border around the slice image.

---

11. File: RandomRotator.cs

General Purpose

Utility component for testing. Rotates its GameObject continuously at a random speed on all three axes. Used to create dynamic objects for testing the per-frame voxelisation pipeline.

```
void OnEnable()
{
    _rotSpeed = new Vector3(
        Random.Range(-maxSpeed, maxSpeed),
        Random.Range(-maxSpeed, maxSpeed),
        Random.Range(-maxSpeed, maxSpeed)
    );
}

void Update()
{
    transform.Rotate(_rotSpeed * Time.deltaTime, Space.Self);
}
```

Unity API: `transform.Rotate(eulers, relativeTo)` — Applies incremental rotation. `Space.Self` means the rotation is in the object's local coordinate system (so the object "tumbles" rather than orbiting a world axis). `Time.deltaTime` makes the rotation frame-rate independent.

---

12. Cross-Cutting Concepts

Race Conditions in GPU Voxelisation

When multiple triangles overlap the same voxel, their `Surface` kernel threads write to the same buffer location simultaneously. Three approaches exist:

1. **Atomic operations** (used here) — `InterlockedOr` is safe but limited to integer types. For a simple 0/1 fill flag, this is perfect.
2. **RWTexture3D writes** — If normals were written per-triangle to `RWTexture3D<float4>`, the last writer wins (benign race). The repo's earlier experiments showed this produces "rainbow garbage" artefacts due to partially-written float4 values.
3. **Per-triangle locking** — Not practical on GPU hardware.

The project chose approach 1 for surface marking and approach 2's failure is documented in the repo memory as a lesson learned.

Why StructuredBuffer for Voxelisation but Texture3D for Ray March

- `StructuredBuffer<uint>` supports atomic operations (`InterlockedOr`), which `RWTexture3D<float>` does NOT support for float types. Atomics are essential for safe concurrent triangle processing.
- `Texture3D` has hardware-accelerated lookup (`Load()`) with single-clock texture cache hits. The ray march benefits from this because nearby pixels cast similar rays, creating spatial locality in texture accesses.

The `BuildTexture` kernel bridges these two representations by copying from the buffer to the texture.

Signed vs Unsigned Fill

This system uses **unsigned** fill (0 = empty, 1 = filled). Unlike the VoxelSDF system in the same repo (which computes signed distance fields with positive/negative values), the VoxelTracer treats the volume as purely binary. This simplifies the pipeline but means:

- No smooth distance-based effects (soft shadows, ambient occlusion)
- No sub-voxel surface positioning
- Interior detection relies on flood fill rather than sign

Thread Group Size Choices

Kernel	Thread Group	Rationale
3D spatial (Clear, BuildTex, Blur, Normals)	(8,8,8) = 512	Standard for 3D grids. Matches GPU warp granularity. Good 3D cache locality.
Linear (CopyStatic, ClearFlood)	(256,1,1) = 256	Maximises occupancy for 1D sequential memory access.
Per-triangle (Surface)	(64,1,1) = 64	Moderate size because each thread has variable work (SAT loop over voxel AABB). Higher thread counts risk register pressure.
Per-line sweep (SweepFill)	(8,8,1) = 64	2D dispatch over perpendicular plane. 8×8 gives good occupancy with minimal wasted threads.
Per-pixel (RayMarch)	(8,8,1) = 64	Standard for full-screen passes. 8×8 tiles benefit from texture cache locality.

13. References & Plagiarism Analysis

This section identifies the academic sources, open-source projects, and established algorithms that the VoxelTracer code draws from, highlights specific code similarities, and describes what makes this project's implementation unique.

13.1 Academic & Algorithm References

Separating Axis Theorem (SAT) Triangle-AABB Test

Source: Tomas Akenine-Möller, "Fast 3D Triangle-Box Overlap Testing" (Journal of Graphics Tools, 2001).

- **Paper:** Describes exactly the 13-axis SAT test used in `TriAABBIntersect()`.
- **Code similarity:** The structure of testing 9 cross-product axes, 3 AABB axes, and 1 triangle normal axis is identical to Möller's reference implementation. The `AxisOverlap()` helper function mirrors the projection-and-compare pattern from the paper.
- **Difference:** The code pre-expands cross products inline (e.g., `float3(0, -f0.z, f0.y)`) rather than using a general cross-product function, which is a common optimisation.

3D DDA Ray Traversal

Source: John Amanatides and Andrew Woo, "A Fast Voxel Traversal Algorithm for Ray Tracing" (Eurographics, 1987).

- **Paper:** Defines the `tMax`/`tDelta`/`stepDir` formulation used in `RayMarch()`.
- **Code similarity:** The DDA loop structure (finding the axis with minimum tMax, stepping, incrementing tMax by tDelta) is a direct implementation of the Amanatides-Woo algorithm. This is the canonical implementation used across almost all voxel ray tracers.
- **Difference:** The normal-from-DDA-step-axis technique (tracking `lastAxis`) is a well-known extension not in the original paper but appearing in numerous voxel engine tutorials.

Jump Flood Algorithm (JFA)

Source: Guodong Rong and Tiow-Seng Tan, "Jump Flooding in GPU with Applications to Voronoi Diagram and Distance Transform" (I3D, 2006).

- **Usage:** The sister file `VoxelSDF.compute` uses JFA; the VoxelTracer itself does NOT use JFA (it uses sweep flood fill instead).
- **Relevance:** The sweep flood fill is a custom, simpler alternative that trades accuracy for lower complexity.

Closest Point on Triangle

Source: Christer Ericson, "Real-Time Collision Detection" (Morgan Kaufmann, 2004), Section 5.1.5.

- **Usage:** The `ClosestPointOnTriangle()` function in `VoxelSDF.compute` uses Ericson's barycentric region-testing method.
- **Code similarity:** The function is a near-direct translation of Ericson's pseudocode, using the `d1`–`d6` variable naming convention.

Ray-AABB Slab Method

Source: Amy Williams et al., "An Efficient and Robust Ray–Box Intersection Algorithm" (JCGT, 2005); originally from Kay-Kajiya (1986).

- **Code similarity:** The `RayAABB()` function uses the standard slab-method formulation found in virtually every ray tracing textbook.

13.2 Open-Source Code References

mattatz/unity-voxel (GitHub)

Repository: `mattatz/unity-voxel` — A well-known Unity GPU voxeliser using compute shaders.

- **Similarities:**
  - The tri struct layout (`float3 a, b, c, normal`) and its transfer via `StructuredBuffer` follows the same pattern.
  - The concept of surface voxelisation followed by a fill pass is shared.
  - The `CPUVoxelizer.cs` file in the `Assets/Scripts/` folder (separate from VoxelTracer) contains a `Voxel_t` struct with `fill`/`front` fields and a `frontFacing` classification — this closely matches the mattatz approach of distinguishing front-face and back-face surface voxels for Z-scan filling.
  - The `Intersects()` function called in `CPUVoxelizer.cs` is the SAT triangle-AABB test, same as mattatz's.
- **Key differences in VoxelTracer:**
  - mattatz uses separate front-face/back-face surface passes and a Z-scan fill. VoxelTracer uses a single `Surface` kernel (no front/back distinction) with a 6-direction sweep flood fill — a fundamentally different interior detection approach.
  - mattatz has no static/dynamic split. VoxelTracer caches static geometry and only re-voxelises dynamic objects.
  - mattatz does not have a DDA ray march renderer — it generates mesh geometry from voxels. VoxelTracer renders directly via ray marching.
  - mattatz voxelises individual meshes in object space. VoxelTracer voxelises the entire scene in world space.

SebLague/Voxel-Rendering (YouTube/GitHub)

Author: Sebastian Lague — popular YouTube educator on voxel rendering and ray marching.

- **Similarities:**
  - The concept of dispatching a compute shader per screen pixel for DDA ray marching is a standard technique demonstrated in Lague's videos.
  - Using `OnRenderImage` for compositing is a common Unity pattern shown in many tutorials.
- **Key differences:**
  - Lague's implementations typically focus on infinite terrain/chunk systems with SDF-based ray marching. VoxelTracer uses a bounded grid with binary fill and DDA traversal.
  - No static/dynamic split or flood fill in Lague's work.

Scrawk/GPU-Voxelization (GitHub)

Repository: `Scrawk/GPU-Voxelization` — Unity GPU voxeliser.

- **Similarities:**
  - Uses compute shaders for triangle-AABB SAT voxelisation.
  - Similar buffer/texture architecture (StructuredBuffer for voxelisation, Texture3D for output).
- **Key differences:**
  - Scrawk uses rasterisation-based conservative voxelisation in some modes, not pure SAT compute.
  - No DDA ray march rendering.
  - No flood fill or interior detection.

13.3 Unity Standard Patterns

Many code patterns in this project are standard Unity practices used across thousands of projects:

- `FindObjectsByType<T>()` for scene queries
- `OnRenderImage` + `Graphics.Blit` for post-processing
- `ComputeBuffer` / `RenderTexture` lifecycle management (allocate in OnEnable, release in OnDisable)
- `[ContextMenu("...")]` for editor quick-actions
- `[Range]`, `[Min]`, `[Header]`, `[Tooltip]` inspector attributes
- `Matrix4x4.MultiplyPoint3x4()` for mesh vertex transformation
- `SkinnedMeshRenderer.BakeMesh()` for snapshot of animated meshes

13.4 What Makes This Project Unique

The following aspects are either novel combinations or original implementations not found in the referenced projects:

1. **Static/Dynamic Split Architecture:** The dual-buffer caching system with `_staticVoxelBuf` → `CopyStaticToWorking` → dynamic overlay is not present in mattatz, Scrawk, or standard tutorials. This is a custom optimisation for real-time scenes with mixed static/dynamic geometry.

2. **6-Direction Sweep Flood Fill:** The `SweepFill` kernel with its bidirectional sweep and cross-axis propagation restart (`|| _FloodBuffer[idx] > 0u`) is a custom algorithm. It's simpler and faster than a true BFS flood fill on GPU but less general. This specific formulation (forward + backward per axis with inter-axis information sharing) is original.

3. **Blur-then-Gradient Normal Pipeline:** While gradient-from-SDF normals are standard, applying a 3D box blur to a *binary fill field* before computing the gradient is an uncommon approach. The standard technique is to use an SDF and compute gradient normals directly from distance values. Using blur on a binary field to create a "pseudo-SDF" for normal estimation is a creative workaround that avoids needing an actual SDF.

4. **DDA Face-Crossing Normals for Rendering:** While face-crossing normals are a known DDA extension, using them as the PRIMARY rendering normals (not the gradient normals from the fill texture) is an intentional design choice for clean, artifact-free voxel rendering. The gradient normals exist only for the debug gizmo.

5. **Terrain Bottom-Surface Generation:** Adding reversed-winding bottom-face triangles to terrain heightmaps (creating a closed volume) to make the sweep flood fill work correctly with open surfaces is a practical solution not commonly documented.

6. **Integrated Scene-Wide Voxelisation:** The automatic discovery and voxelisation of ALL scene geometry types (MeshFilter, SkinnedMeshRenderer, Terrain) in a single unified pipeline, with per-object dynamic tagging via `VoxelDynamic`, is a more comprehensive approach than most open-source voxelisers which operate on individual meshes.

7. **Component-Based Dynamic Classification:** Using `VoxelDynamic` as a marker component for runtime classification is a clean Unity-idiomatic approach that allows per-object control without modifying the voxeliser code.

13.5 Summary Table

Code Element	Source/Reference	Similarity Level	Unique Aspects
`TriAABBIntersect()`	Akenine-Möller (2001)	High — standard algorithm	Inline cross-product expansion
`RayMarch()` DDA	Amanatides-Woo (1987)	High — standard algorithm	Combined with DDA face normals
`RayAABB()`	Kay-Kajiya / Williams (1986/2005)	Exact — textbook method	None (universal utility)
`SweepFill`	Custom	Low — novel approach	Bidirectional + cross-axis propagation
Static/Dynamic split	Custom	Low — no direct reference	Unique caching architecture
`BlurFill` + `ComputeNormals`	Custom	Low — unusual technique	Binary blur as pseudo-SDF
`AppendTerrain()` bottom faces	Custom	None — practical invention	Closes open terrain for flood fill
`CPUVoxelizer.cs`	mattatz/unity-voxel	High — similar structure	In Scripts/ folder, not part of VoxelTracer
Composite shaders	Standard Unity pattern	Generic	Depth-aware variant adds value
`VoxelDynamic` marker	Unity component pattern	Generic	Applied to voxel classification

End of walkthrough document.